
MARIE: wprowadzenie do budowy prostego komputera

„Jeżeli do uzyskania wyniku musisz posłużyć się jakimś instrumentem, budując go, nie pozwól sobie na to, aby był on zbyt skomplikowany”.

— Leonardo da Vinci

4.1. Wstęp

W dzisiejszych czasach zaprojektowanie komputera to zadanie dla dobrze wyszkolonego informatyka. Niemożliwością jest, aby w książce takiej jak ta (ani w podstawowym kursie o architekturze komputera) przedstawić wszystko, co jest niezbędne do zbudowania współczesnego komputera. Tym niemniej w niniejszym rozdziale przyjrzymy się bardzo prostemu komputerowi, jakim jest MARIE (ang. *A Machine Architecture that is Really Intuitive and Easy*), co w języku polskim oznacza naprawdę prostą i intuicyjną architekturę komputera. Następnie krótko omówimy komputery Intel i MIPS — dwie popularne architektury, będące odzwierciedleniem filozofii projektowania CISC oraz RISC. Celem niniejszego rozdziału jest wyjaśnienie, jak naprawdę działa komputer. Dlatego też staraliśmy się, zgodnie z zacytowaną radą Leonarda da Vinci, przedstawić budowę komputera w możliwie najmniej skomplikowany sposób.

4.1.1. Podstawy budowy i organizacji procesora

Z rozdziału 2. („Reprezentacja danych w systemach komputerowych”) wiemy już, że komputer musi operować na danych zapisanych w systemie binarnym. Z rozdziału 3. z kolei dowiedzieliśmy się, że pamięć wykorzystywana jest do przechowywania zarówno danych, jak i poleceń programów (również zapisanych w systemie binarnym). Programy muszą być w jakiś sposób wykonywane, a dane przetwarzane. Jest to zadanie *procesora* (ang. *central processing unit, CPU*), który jest odpowiedzialny za wczytywanie poleceń programu, rozkodowanie każdego z nich oraz wykonanie działań we wskazanej kolejności. Aby zrozumieć, jak pracują komputery, należy najpierw zaznajomić się z ich różnorodnymi podsystemami i interakcjami pomiędzy nimi. Zanim przedstawimy prostą architekturę komputera, przyjrzymy się najpierw mikroarchitekturze występującej na poziomie sterowania współczesnych komputerów.

We wszystkich komputerach znajduje się procesor. Może on być podzielony na dwie części. Pierwszą z nich jest *ścieżka danych*, czyli sieć jednostek przechowujących dane (rejestrów) oraz jednostek logiczno-arytmetycznych (do wykonywania różnych operacji na danych), połączonych ze sobą magistralami (zdolnymi do przenoszenia danych z miejsca na miejsce), których pracą sterują zegary. Drugim składnikiem procesora jest *jednostka sterująca* — moduł odpowiedzialny za ustalanie kolejności wykonywanych operacji i gwarantujący, że dane znajdują się we właściwym miejscu o właściwym czasie. Łącznie oba te składniki wykonują zadania powierzone procesorowi: wczytywanie instrukcji, rozkodowywanie ich i wykonywanie ich we wskazanej kolejności. Działanie komputera zależy bezpośrednio od sposobu zaprojektowania ścieżki danych i jednostki sterującej. Dlatego poniżej zostaną one szczegółowo omówione.

Rejestry

Rejestry są wykorzystywane w systemach komputerowych w charakterze miejsc, w których przechowywane są różnego rodzaju dane, takie jak adresy, liczniki rozkazów lub dane niezbędne do wykonania określonego programu. Mówiąc prościej, *rejestr* jest fizycznym urządzeniem, które przechowuje dane binarne. Aby dostęp do nich był możliwie szybki, rejestry umiejscowione są w procesorze. W rozdziale 3. dowiedziałeś się, że do zbudowania rejestrów można użyć przerzutników D. Jeden przerzutnik D odpowiada rejestrowi przechowującemu 1 bit. Zatem do przechowywania wartości wielobitowych niezbędny jest zespół przerzutników D. Na przykład aby zbudować rejestr szesnastobitowy, musielibyśmy połączyć ze sobą 16 przerzutników D. Na rysunku licznika binarnego, który zamieściliśmy w rozdziale 3., można było zauważyć, że wspomniane przerzutniki muszą być zsynchronizowane tak, aby działały jednocześnie. Z każdym impulsem zegara dane wchodzi do rejestru i nie mogą być zmienione, dopóki nie nastąpi kolejny impuls (są tam więc przechowywane).

Przetwarzanie danych na komputerze jest zazwyczaj wykonywane na słowach binarnych o określonej wielkości, które są przechowywane w rejestrach. Dlatego większość komputerów ma rejestry o określonej pojemności. Popularne rozmiary rejestrów to 16, 32 i 64 bity. Liczba rejestrów w komputerze różni się w zależności od architektury, ale zazwyczaj jest potęgą liczby dwa, a najpopularniejsze z nich to 16 i 32. Rejestry zawierają dane, adresy lub informacje sterujące. Niektóre z nich są wyspecjalizowane, a więc mogą zawierać tylko dane, tylko adresy lub tylko informacje sterujące. Inne są bardziej ogólne i mogą przechowywać w różnych momentach zarówno dane, adresy, jak i informacje sterujące.

Dane są w rejestrach zapisywane, z nich odczytywane i przenoszone z rejestru do rejestru. Do rejestrów nie odwołujemy się tak samo, jak do pamięci (przypominamy, że każde słowo w pamięci ma swój niepowtarzalny adres, liczony od zera). To sama jednostka sterująca odwołuje się do nich i nimi manipuluje.

We współczesnych systemach komputerowych istnieje wiele rodzajów wyspecjalizowanych rejestrów: rejestry do przechowywania informacji, rejestry do zmieniania wartości, rejestry do porównywania wartości oraz rejestry do liczenia. Są też rejestry zwane „brudnopisami”, które przechowują tymczasowe wartości, rejestry indeksowe do kontrolowania pętli, rejestry do zarządzania stosami informacji dla procesów, rejestry stanu do przechowywania informacji o stanie lub trybie pracy (i występowaniu takich zdarzeń, jak przepełnienie, przeniesienie czy osiągnięcie warunków zerowych) oraz rejestry ogólnego przeznaczenia, czyli rejestry dostępne dla programisty. W większości komputerów znajdują się zespoły rejestrów, przy czym każdy z tych zespołów jest zespołem wyspecjalizowanym. Na przykład w architekturze Pentium istnieje zespół rejestrów danych i zespół rejestrów adresowych. Niektóre architektury mają bardzo duże zespoły rejestrów, które mogą być w nowoczesny sposób wykorzystywane do przyspieszenia procesu wykonywania operacji. (Temat ten omówimy po przedstawieniu zaawansowanych architektur w rozdziale 9.).

ALU

Jednostka arytmetyczno-logiczna (ALU) wykonuje operacje logiczne (takie, jak na przykład porównanie) i arytmetyczne (na przykład dodawanie i mnożenie) wymagane podczas wykonywania programu. Przykład prostej jednostki ALU widzieliśmy już w rozdziale 3. Ogólnie mówiąc, jednostka ALU ma dwa wejścia danych i jedno wyjście. Operacje wykonywane w ALU mają często wpływ na bity w *rejestrze stanu* (zadaniem tych bitów jest informowanie o wystąpieniu określonych sytuacji, takich jak na przykład przepełnienie). Jednostka ALU „wie”, które operacje wykonywać, ponieważ jest kontrolowana przez sygnały płynące z jednostki sterującej.

Jednostka sterująca

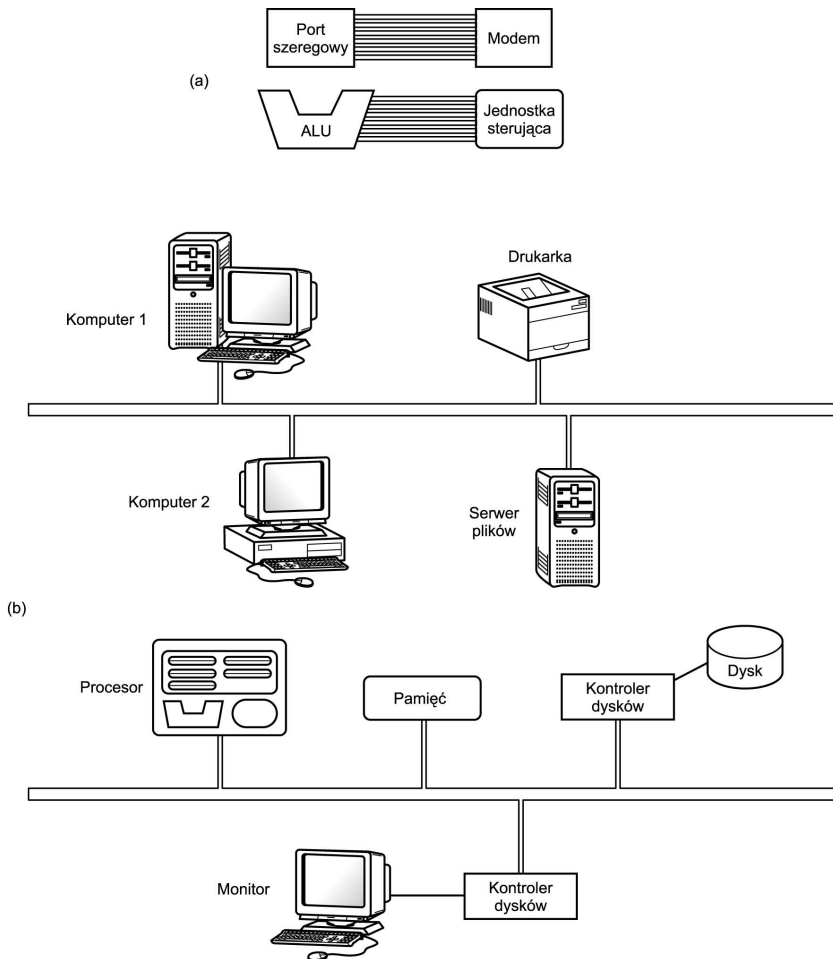
Jednostka sterująca jest „policjantem” czy też „kierującym ruchem” procesora. Monitoruje ona wykonywanie wszystkich instrukcji oraz transfer wszystkich informacji. Wydobywa instrukcje z pamięci, rozkodowuje je i dba o to, aby znalazły się w odpowiednim czasie we właściwym miejscu. Informuje też jednostkę ALU, których rejestrów ma ona użyć, obsługuje przerwania oraz — w celu wykonania pożądanej operacji — włącza odpowiedni zestaw obwodów elektrycznych w ALU. Aby odnaleźć następną instrukcję, jaką należy wykonać, jednostka sterująca wykorzystuje rejestr *licznika rozkazów*, natomiast aby „wiedzieć” na bieżąco o przepełnieniach, przesunięciach, „zapożyczeniach”, itp. — używa rejestru stanu. Więcej szczegółowych informacji o jednostce sterującej prezentujemy w podrozdziale 4.7.

4.1.2. Magistrala

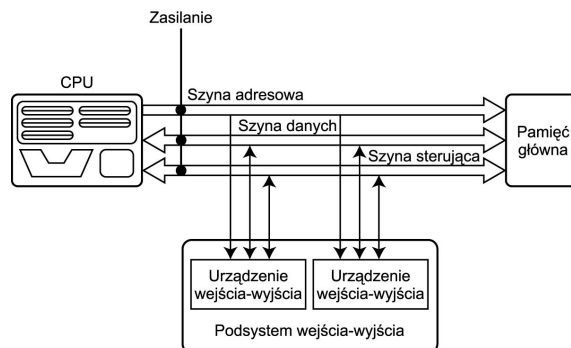
Procesor komunikuje się z innymi podsystemami poprzez tzw. szynę (zwaną też magistralą). *Magistrala* jest zespołem przewodów działających jako wspólna ścieżka danych, łącząca różnorodne podsystemy. Składa się z wielu linii, co pozwala na równoległy przepływ bitów. Szyny są niedrogim, ale też wszechstronnym sposobem na łatwe włączenie nowych urządzeń do systemu i połączenie ich ze sobą. W danej chwili czasu szyna może być wykorzystywana tylko przez jedno urządzenie, niezależnie od tego, czy jest to rejestr, jednostka ALU, pamięć czy cokolwiek innego. Pomimo to współużytkowanie szyny często staje się „wąskim gardłem” w komunikacji. Na prędkość szyny wpływa zarówno jej szerokość, jak i liczba urządzeń, przez które jest współużytkowana. Dość często urządzenia te podzielone są na dwie kategorie: urządzenia *master* i urządzenia *slave*, przy czym urządzenie *master* to takie, które inicjuje działanie, natomiast urządzenie *slave* odpowiada na zapytania urządzenia *master*.

Magistrala może być dwupunktowa, co oznacza, że łączy ona dwa konkretne podzespoły (tak, jak to zostało przedstawione na rysunku 4.1a), lub też może być *wspólną ścieżką* danych, która łączy kilka urządzeń, zmuszając je do jej współużytkowania (nazywamy ją magistralą *wielopunktową*, patrz rysunek 4.1b).

W związku ze wspomnianym współużytkowaniem niezwykle ważny jest *protokół magistrali* (zestaw reguł użytkowania). Rysunek 4.2 pokazuje typową magistralę, składającą się z linii danych, linii adresowych, linii sterujących i linii zasilania. Często linie magistrali odpowiedzialne za przenoszenie danych nazywane są *szyną danych*. Linie te zawierają faktyczne informacje, które muszą zostać przeniesione z jednego miejsca do innego. *Linie sterujące* wskazują, które z urządzeń ma pozwolenie na użycie szyny i w jakim celu (np. zapis lub odczyt danych z pamięci bądź z urządzenia wejścia-wyjścia). Linie sterujące przesyłają też komunikaty potwierdzające odbieranie żądań dostępu do magistrali, żądań obsługi przerwań i impulsów synchronizacyjnych zegara). *Linie*



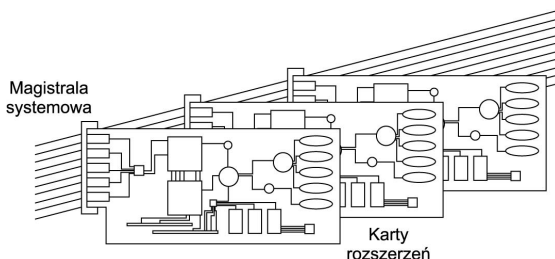
RYSUNEK 4.1. a) szyny dwupunktowe; b) szyna wielopunktowa



RYSUNEK 4.2. Budowa typowej magistrali

adresowe wskazują miejsce (np. w pamięci), w którym dane powinny zostać zapisane lub z którego należy je odczytać. *Linie zasilania* zapewniają niezbędne zasilanie elektryczne. Typowe operacje w przypadku magistrali to wysyłanie adresu (do odczytu lub zapisu), transfer danych z pamięci do rejestru (odczytanie pamięci) oraz transfer danych z rejestru do pamięci (zapis w pamięci). Dodatkowo magistrale wykorzystywane są do operacji zapisu i odczytu danych z urządzeń peryferyjnych. Każdy rodzaj transferu ma miejsce w cyklu magistrali, to znaczy w okresie czasu pomiędzy dwoma impulsami zegara szyny.

Ze względu na różne rodzaje szyn transportujących informacje oraz ze względu na różnorodność urządzeń, które te szyny wykorzystują, również magistrale możemy podzielić na kilka rodzajów. Magistrale *procesor-pamięć* są krótkimi, bardzo szybkimi szynami, ściśle dopasowanymi do zastosowanego w komputerze systemu pamięci. Stosowane są w celu maksymalnego powiększenia przepustowości (szybkości przesyłu danych) i zazwyczaj mają dość specyficzną budowę. *Magistrale wejścia-wyjścia* są zazwyczaj dłuższe niż poprzednie i łączą wiele typów urządzeń o różnej przepustowości. Budowa tych szyn jest kompatybilna z różnymi architekturami komputerów. *Magistrala systemowa* (rysunek 4.3) jest właściwie wbudowana w płytę główną komputera. Łączy ona procesor, urządzenia wejścia-wyjścia oraz pamięć (tak więc wszystkie te urządzenia współdzielą jedną szynę). Wiele komputerów ma pewną hierarchię magistral, stąd też występowanie dwóch magistral (na przykład szyny procesor-pamięć i szyny wejścia-wyjścia) lub ich większej liczby wewnątrz jednego systemu nie jest niczym nadzwyczajnym. Systemy o dużej wydajności często używają wszystkich trzech rodzajów magistral.



RYSUNEK 4.3. Magistrala systemowa

W przypadku komputerów PC stosujemy nieco odmienną terminologię. Komputery te mają wewnętrzną szynę zwaną *magistralą systemową*, która łączy procesor, pamięć i wszystkie inne wewnętrzne podsystemy. Szyny zewnętrzne (określane często mianem *szyn rozszerzeń*) łączą urządzenia zewnętrzne, peryferyjne, gniazda rozszerzeń oraz porty wejścia-wyjścia z resztą komputera. W większości komputerów PC znajdują się również *szyny lokalne*, czyli szyny danych, które łączą urządzenie peryferyjne bezpośrednio z procesorem. Są to bardzo szybkie magistrale i mogą być wykorzystane do połączenia tylko ograniczonej liczby podobnych urządzeń. Magistrale rozszerzeń są wolniejsze, ale umożliwiają bardziej uniwersalne połączenia. Rozdział 7. bardzo szczegółowo opisuje te zagadnienia.

Magistrale to w rzeczywistości niewiele więcej aniżeli wiązki przewodów, których działanie usankcjonowane jest pewnymi standardami odnośnie połączeń, taktowania, przesyłania impulsów i wykorzystywanego protokołu przesyłu danych. Szyny *synchroniczne* są zsynchronizowane w taki sposób, że transakcje zachodzą w nich tylko w czasie trwania impulsu (taktu) zegara (kolejność zdarzeń kontrolowana jest przez zegar). Każde urządzenie zsynchronizowane jest z częstotliwością, z jaką zegar wysyła impuls. Wspomniany wyżej czas trwania impulsu zegara stanowi odwrotność jego *częstotliwości*. Na przykład jeżeli częstotliwość taktowania zegara magistrali wynosi 133 MHz,

to czas trwania jednego taktu wynosi $1/133\,000\,000$, czyli 7,52 ns. Ponieważ zegar steruje transakcjami, jakakolwiek *rozbieżność zegarów* (odchylenie) może potencjalnie spowodować problemy, co oznacza że szyna musi być możliwie najkrótsza, aby odchylenie zegara nie mogło być zbyt duże. Ponadto czas trwania cyklu magistrali nie może być krótszy niż okres czasu potrzebny do przetransportowania danych przez szynę. A zatem długość magistrali stawia pewne ograniczenia zarówno dla częstotliwości taktowania magistrali, jak i dla czasu trwania cyklu magistrali.

W przypadku magistral *asynchronicznych* to linie sterujące koordynują operacje. Jednocześnie w celu zapewnienia właściwego odliczania czasu zastosowany musi być tu skomplikowany *protokół uzgadniania*. Na przykład aby możliwe było odczytanie słowa danych z pamięci, w protokole tym konieczne byłoby wykonanie mniej więcej następujących kroków:

- (1) ReqREAD: ta linia sterująca magistrali jest uaktywniana i w tym samym czasie na odpowiedniej linii szyny umieszczony jest adres danych zapisanych w pamięci.
- (2) ReadyDATA: ta linia sterująca magistrali jest aktywowana, gdy podsystem pamięci przygotowuje dane dla magistrali i umieści je na linii danych.
- (3) ACK: ta linia sterująca magistrali używana jest w celu potwierdzenia wykonania operacji ReqREAD lub ReadyDATA.

Dzięki sterowaniu transakcjami za pośrednictwem specjalnego protokołu (a nie zegara) nowoczesne magistrale coraz lepiej dzielą swój czas między korzystające z nich urządzenia i mogą współpracować z większą ich liczbą.

Aby wykorzystać magistralę, urządzenie musi sobie ją zarezerwować, ponieważ tylko jedno urządzenie może używać szyny w tym samym momencie. Jak już zostało to wspomniane, urządzenia pracujące w trybie bus master mogą same zainicjować transfer informacji (kontrolować magistralę), podczas gdy urządzenia typu slave są aktywowane przez nadrzędne wobec nich urządzenia master i odpowiadają jedynie na żądania odczytu lub zapisu danych (tak więc tylko urządzenia master mogą zarezerwować magistralę). Obydwa rodzaje urządzeń działają w zgodzie z protokołem komunikacyjnym i muszą wywiązywać się ze swych zadań w bardzo ściśle określonych przedziałach czasowych. W bardzo prostym systemie (takim, jaki prezentujemy w następnym podrozdziale) jedynym urządzeniem mogącym pracować w trybie master jest procesor. Pozwala to uniknąć chaosu na magistrali, ale zauważmy, że przy takim rozwiązaniu procesor jest włączony w przebieg każdej operacji wykorzystującej szynę.

W systemach z więcej niż jednym urządzeniem master wymagany jest arbitraż magistrali. Mechanizmy arbitrażu magistrali muszą przyznawać priorytet określonym urządzeniom master i jednocześnie gwarantować, że urządzenia o niższym priorytecie nie zostaną całkowicie zablokowane. Mechanizmy *arbitrażu magistrali* możemy podzielić na cztery kategorie:

- (1) **Arbitraż szeregowy:** metoda ta wykorzystuje linię sterującą „przydziel magistralę”, która jest „przekazywana” od urządzenia o najwyższym priorytecie do tego o najniższym. Nie ma tu jednak sprawiedliwości i możliwa jest sytuacja, w której urządzenia o niższym priorytecie zostaną zablokowane i magistrala nigdy nie zostanie im udostępniona. Metoda ta jest więc prosta, ale niesprawiedliwa.
- (2) **Scentralizowany arbitraż równoległy:** każde urządzenie ma linię sterującą żądającą dostępu do magistrali, a centralny arbiter wybiera, które urządzenie go otrzyma. W przypadku arbitrażu tego typu mogą wystąpić „zatory” komunikacyjne.
- (3) **Zdecentralizowany arbitraż oparty na samodzielnym wyborze:** schemat ten podobny jest do arbitrażu zcentralizowanego, ale zamiast centralnej jednostki podejmującej decyzję urządzenia same określają, które z nich ma wyższy priorytet i powinno otrzymać dostęp do magistrali jako pierwsze.

- (4) **Zdecentralizowany arbitraż z wykrywaniem kolizji:** każde urządzenie może zażądać dostępu do magistrali. Jeśli zaś magistrala wykryje jakiegokolwiek kolizję (wiele równoczesnych żądań), urządzenie to będzie musiało wystosować żądanie jeszcze raz. (Ten typ arbitrażu stosowany jest w sieciach Ethernet).

Rozdział 7. zawiera więcej szczegółowych informacji na temat magistrali i ich protokołów.

4.1.3. Zegary

Każdy komputer ma wewnętrzny zegar, determinujący szybkość wykonywania poleceń. Zegar synchronizuje również wszystkie podzespoły systemu. Poprzez nadanie impulsu ustawia on tempo dla wszystkich operacji, które mają miejsce w systemie — w podobny sposób, jak metronom czy też dyrygent orkiestry. Procesor wykorzystuje zegar do regulowania swojej pracy, kontrolując szybkość wykonywania przez bramki logiczne określonych operacji. Wykonanie każdej instrukcji zabiera procesorowi określoną liczbę impulsów zegara. Stąd też czas wykonania polecenia jest często mierzony w *cyklach zegara* — czyli czasie, jaki upływa między jego dwoma impulsami — a nie w sekundach. *Częstotliwość zegara* (czasami nazywana też jego szybkością) mierzona jest w megahercach. Jak już wiemy z rozdziału 1., 1 MHz oznacza milion cykli na sekundę (tak więc 1 herc to 1 cykl na sekundę). *Czas trwania cyklu zegara* (inaczej *okres zegara*) jest po prostu odwrotnością częstotliwości zegara. Na przykład 800-megahercowy komputer ma czas trwania cyklu zegara równy 1/800 000 000 s, a więc 1,25 ns. Jeśli zaś cykl zegara trwa w komputerze 2 ns, to wiemy, że jest on taktowany częstotliwością 500 MHz.

Większość komputerów jest synchroniczna, to znaczy główny impuls zegara jest w nich nadawany w równych odstępach czasu (a jego wartość zmienia się cyklicznie z 0 na 1 i z powrotem). Rejestry muszą czekać na impuls zegara, zanim załadują nowe dane. Rozsądnie byłoby przypuszczać, że jeśli przyspieszymy zegar, komputer będzie pracował szybciej. Istnieją jednak ograniczenia co do tego, jak krótki może być cykl zegara. Kiedy zegar wysyła impuls i do rejestrów ładowane są nowe dane, dane wynikowe rejestru z dużym prawdopodobieństwem się zmieniają. Te zmienione wartości muszą przejść przez wszystkie układy w komputerze, zanim dotrą do wejścia następnego zespołu rejestrów, w których będą przechowywane. Cykl zegara musi być na tyle długi, aby zmienione dane mogły dotrzeć do następnego zespołu rejestrów. Jeżeli byłby on zbyt krótki, niektóre wartości mogłyby nie dotrzeć do rejestrów. Spowodowałoby to powstanie pewnej niespójności, a tego chcemy za wszelką cenę uniknąć. Stąd też najkrótszy cykl zegara musi trwać przynajmniej tyle, ile wynosi maksymalne opóźnienie przesyłu danych z każdego zespołu rejestrów wynikowych do rejestrów wejściowych. Co stanie się, jeśli skrócimy odległość pomiędzy rejestrami, aby ograniczyć opóźnienie w rozprzestrzenianiu się danych? Moglibyśmy tego dokonać poprzez dodanie rejestrów pomiędzy rejestrami wynikowymi i odpowiadającymi im rejestrami wejściowymi. Przypomnijmy jednak, że rejestry nie mogą zmienić wartości, zanim nie pojawi się impuls zegara. W efekcie otrzymamy więc zwiększoną liczbę cykli zegara. Na przykład polecenie, które pierwotnie wymagało 2 cykli zegara, będzie teraz wymagało trzech albo czterech cykli (lub nawet więcej — w zależności od tego, gdzie umieścimy dodatkowe rejestry).

Większość poleceń komputera wymaga 1 lub 2 cykli zegara, ale niektóre mogą trwać nawet 35 lub więcej. Poniższy wzór pokazuje zależność między sekundami a cyklami zegara:

$$\text{czas_procesora} = \frac{\text{liczba_sekund}}{\text{program}} = \frac{\text{liczba_instrukcji}}{\text{program}} * \frac{\text{\textit{średnia_liczba_cykli}}}{\text{instrukcję}} * \frac{\text{liczba_sekund}}{\text{cykl}}$$

Ważne jest, abyśmy zauważyli, że architektura komputera ma ogromny wpływ na jej wydajność. Dwa komputery o tych samych prędkościach zegarów niekoniecznie będą kończyły wykonywanie poleceń w ciągu tej samej liczby cykli. Na przykład operacja mnożenia wykonywana przez starszy model Intel 286 wymagała 20 cykli zegara, ale już w przypadku nowoczesnego Pentium może być wykonana w ciągu jednego cyklu, co wskazuje na to, że nowy komputer jest 20 razy szybszy niż 286, nawet gdyby były one taktowane takim samym zegarem. W zasadzie mnożenie wymaga więcej czasu aniżeli dodawanie, obliczenia na liczbach zmiennoprzecinkowych trwają dłużej niż na liczbach całkowitych, a sięgnięcie do pamięci zabiera więcej czasu niż do rejestrów.

Generalnie, jeśli odwołujemy się do terminu *zegar*, mamy na myśli zegar systemowy, czyli nadrzędny zegar regulujący funkcjonowanie procesora i innych komponentów. Jednakże niektóre magistrale mają swoje własne zegary. *Zegary magistral* są zazwyczaj wolniejsze od zegarów procesora, co powoduje problem „wąskiego gardła”.

Poszczególne podzespoły komputera mają narzucone granice czasowe, w których muszą się zmieścić z wykonywaniem swych operacji. Ich producenci gwarantują, że nawet w najgorszej sytuacji sprzęt będzie się trzymał narzuconych „terminów”. Kiedy połączymy razem wszystkie te podzespoły w szereg, w którym jeden z nich musi zakończyć swoje zadanie po to, aby następny mógł prawidłowo działać, ważne jest, abyśmy zdawali sobie sprawę ze wspomnianych granic — wówczas będziemy mogli prawidłowo zsynchronizować urządzenia. Tym niemniej wiele osób forsuje granice niektórych podsystemów, próbując polepszyć wydajność systemu. Jedną z metod przez nie stosowanych jest tzw. *przetaktowywanie*.

Mimo iż możliwe jest przetaktowanie wielu podzespołów, najczęściej zabiegowi temu poddaje się procesor. Podstawowe założenie tego pomysłu to ustawienie zegara procesora i (lub) zegara magistrali powyżej górnej granicy określonej przez producenta. Mimo tego, że taki zabieg może zwiększyć wydajność systemu, trzeba uważać, żeby go nie rozsynchronizować lub nawet gorzej — nie przegrzać procesora. Można też przetaktować magistralę systemową, co spowoduje przetaktowanie wszystkich korzystających z niej podzespołów. Może to polepszyć wydajność komputera, ale może też zniszczyć podłączone do niej urządzenia.

4.1.4. Podsystem wejścia-wyjścia

Urządzenia wejścia-wyjścia (I/O) pozwalają nam na komunikowanie się z systemem komputerowym. Wejście-wyjście to transfer danych pomiędzy pamięcią podstawową a różnymi urządzeniami peryferyjnymi. Urządzenia wejścia — takie jak klawiatury, myszy, czytniki kart, skanery, systemy rozpoznawania głosu oraz ekrany dotykowe — pozwalają nam wprowadzać dane do komputera. Z kolei urządzenia wyjścia — takie jak monitory, drukarki, plotery oraz głośniki — pozwalają nam uzyskać informacje z komputera.

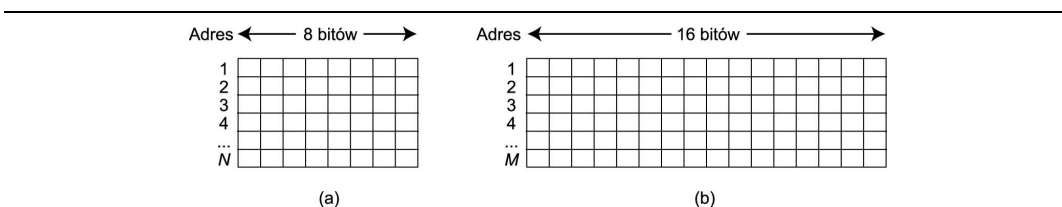
Urządzenia te nie są bezpośrednio połączone z procesorem. W każdym komputerze znajduje się *interfejs*, który zajmuje się transferem danych. Przekształca on sygnały magistrali systemowej na format akceptowany przez dane urządzenie i odwrotnie. Procesor zaś komunikuje się z urządzeniami zewnętrznymi poprzez rejestry wejścia i wyjścia. Taka wymiana danych wykonywana jest na dwa sposoby. Przy *wejściu-wyjściu odwzorowywanym pamięciowo* rejestry interfejsu pojawiają się na mapie pamięci komputera i nie ma w tym przypadku właściwie żadnej różnicy pomiędzy dostępem do pamięci a dostępem do urządzenia wejścia-wyjścia. Jest to wyraźna zaleta, jeśli uwzględniamy prędkość, z tym że takie rozwiązanie zajmuje określoną ilość pamięci w komputerze. W przypadku *wejścia-wyjścia opartego na rozkazach* procesor ma wyspecjalizowane rozkazy realizujące operacje I/O. Pomimo że takie rozwiązanie nie zużywa pamięci, wymaga ono stosowania

specyficznych rozkazów wejścia-wyjścia, co oznacza, że można je stosować tylko w przypadku tych procesorów, które je obsługują. Bardzo istotną rolę w operacjach wejścia-wyjścia odgrywają przerwania. Są one wydajnym sposobem informowania procesora o dostępności operacji wejścia-wyjścia.

4.1.5. Organizacja pamięci i adresowanie

W rozdziale 3. przedstawiliśmy przykład raczej niewielkiej pamięci. Nie omówiliśmy jednak jeszcze szczegółowo tego, jak pamięć jest rozplanowana i jak się do niej odwołujemy. Bardzo ważne jest, abyś rozumiał te dwie kwestie, zanim będziemy kontynuować bieżący temat.

Możemy wyobrazić sobie pamięć jako macierz bitów. Szerokość każdego wiersza, implementowanego w postaci rejestru, jest zazwyczaj taka sama, jak długość słowa maszynowego komputera. Każdy rejestr (nazywany częściej *komórką pamięci*) ma niepowtarzalny adres. Adresy pamięci zazwyczaj zaczynają się od zera i rosną. Rysunek 4.4 przedstawia tę zasadę.



RYSUNEK 4.4. a) N 8-bitowych komórek pamięci, b) M 16-bitowych komórek pamięci

Prawie zawsze adres jest przedstawiony w postaci liczby całkowitej bez znaku. Przypomnijmy sobie z rozdziału 2, że 4 bity to półbajt, a 8 bitów to bajt. Normalnie pamięć jest *adresowana bajtowo*, co oznacza, że każdy indywidualny bajt ma swój niepowtarzalny adres. Niektóre komputery mogą mieć rozmiar słowa większy niż pojedynczy bajt — na przykład komputer może operować 32-bitowymi słowami (co oznacza, że w danej chwili czasu określona instrukcja może modyfikować 32 bity na raz), ale ciągle używać architektury adresowanej bitowo. W sytuacji, kiedy słowo składa się z kilku bajtów, bajt o najniższym adresie decyduje o adresie całego słowa. Jest też możliwe, żeby komputer był *adresowany słowowo*, co oznacza, że każde słowo (niekoniecznie każdy bajt) ma swój własny adres. Tym niemniej większość komputerów adresowana jest bajtowo (nawet jeśli mają 32-bitowe lub większe słowa). Adres pamięci jest zazwyczaj przechowywany w postaci pojedynczego słowa maszynowego.

Jeśli powyższy wywód o komputerach używających adresowania bajtowego i słów o różnej długości nieco Cię zmylił, przydatna może okazać się poniższa analogia. Pamięć podobna jest do ulicy pełnej bloków mieszkalnych. Każdy budynek (słowo) ma wiele mieszkań (bajtów), Zarazem każde mieszkanie ma swój własny adres. Wszystkie mieszkania ponumerowane (adresowane) są w kolejności od 0 do ogólnej liczby mieszkań w kompleksie. Budynki stanowią grupę mieszkań. W komputerach taką samą rolę pełnią słowa. Słowa są podstawową jednostką używaną w różnorodnych rozkazach. Na przykład możesz odczytać lub zapisać słowo w pamięci — nawet na komputerze adresowanym bajtowo.

Jeśli architektura komputera jest adresowana bajtowo, a słowo w rozkazie jest większe niż jeden bajt, należy przeprowadzić *wyrównanie*. Na przykład jeśli chcemy odczytać 32-bitowe słowo na komputerze adresowanym bajtowo, należy upewnić się, że: (1) słowo to zostało zapisane na naturalnej